```
first(df,5) #displaying the first 5 rows to get an overview of
the dataset
```

Figure 1 shows the first five rows of data in the form of a DataFrame. This helps us to understand the data has loaded properly.

3. We can now explore this data set to find out its size, i.e., the number of rows and columns. We can also use the describe method to draw up some statistical data.

```
#checking the number of rows and columns present in the
dataset, that is the shape of the dataset
println(size(df))

#this helps us to check the mean, median, min, max etc.
describe(df)
```

Figure 2 shows the statistical description of the data, i.e., mean, median, min, max, etc.

	variable	mean	min	median	max	nmissing	eltype
	Symbol	Float64	Real	Float64	Real	Int32	DataType
1	Column1	253.5	1	253.5	506	0	Int64
2	crim	3.61352	0.00632	0.25651	88.9762	0	Float64
3	zn	11.3636	0.0	0.0	100.0	0	Float64
4	indus	11.1368	0.46	9.69	27.74	0	Float64
5	chas	0.06917	0	0.0	1	0	Int64
6	nox	0.554695	0.385	0.538	0.871	0	Float64
7	rm	6.28463	3.561	6.2085	8.78	0	Float64
8	age	68.5749	2.9	77.5	100.0	0	Float64
9	dis	3.79504	1.1296	3.20745	12.1265	0	Float64
10	rad	9.54941	1	5.0	24	0	Int64
11	tax	408.237	187	330.0	711	0	Int64
12	ptratio	18.4555	12.6	19.05	22.0	0	Float64
13	black	356.674	0.32	391.44	396.9	0	Float64
14	lstat	12.6531	1.73	11.36	37.97	0	Float64
15	medv	22.5328	5.0	21.2	50.0	0	Float64

Figure 2: Statistical description of the data

4. An important step before implementing the model is to divide the data set into features and target variable. We will take the target variable (house prices) on the Y list and the features on the X.

```
y = df[:, :medv];  #Y-values
X = select!(df, Not(:medv));  #features
```

5. In order to both train and test the model, we will need to divide the data set into training data and testing data. We will use 80 per cent of the data for training and the rest 20 per cent for testing.

```
design_matrix = convert(Matrix, X)
train_size = 0.80 #sepecifying the split ratio
num_samples = size(design_matrix)[1] #total number of samples
in dataset

train_index = trunc(Int, train_size * num_samples)
#truncating 80% of samples for training

# Split using the desired train size
X_train = design_matrix[1:train_index, :]
X_test = design_matrix[train_index+1:end, :]

y_train = y[1:train_index]
y_test = y[train_index+1:end]

print("Dataset split into train and test")
```

6. As a necessary pre-processing step, we will perform scaling and transformation on the training and testing data.

```
# defining function to scale features
function scale_features(X)

    μ = mean(X, dims=1)
    σ = std(X, dims=1)

    X_norm = (X .- μ) ./ σ
    return (X_norm, μ, σ)

end


#defining function to transform features.
function transform_features(X, μ, σ)
    X_norm = (X .- μ) ./ σ
    return X_norm
end

# Scale training features
X_train_scaled, μ, σ = scale_features(X_train)

# Transform the testing features
X_test_scaled = transform_features(X_test, μ, σ)

print("Training and testing data are now transformed")
```

7. We will also need to define a cost function for determining the mean squared error.